

Requisito	Cómo
JWT access+refresh	Cookies HttpOnly
Middleware auth	Tower middleware
SQL injection	SQLx bind params
Password hashing	argon2
Headers CSP/HSTS	middleware
CORS restrictivo	tower-http
Validaciones input	validator
Swagger UI	utoipa

La API corre en localhost (en el puerto 9090), se conecta a una base de datos Postgres llamada **usuarios_rust_db** que está en localhost puerto 5432, con password **"superapostgres"**.

La base de datos ya está creada pero aún no tiene migraciones. Haz las migraciones y crea una tabla de acuerdo a un login, no olvides que también deben existir endpoint de health-check, y endpoint de registro de un usuarios nuevos (con los campos nombre, apellido, user, password).

Usar *Serde* para serialización y deserialización, combinado con crates como validator para validaciones y herramientas como utoipa para OpenAPI. (la idea es tener un endpoint de la documentación donde se puedan probar cada endpoints algo así como <http://localhost:9090/api/v1/swagger-ui/>,

Debes hacer validaciones de cada input del usuario (exige validaciones explícitas) para evitar sql injection.

Usar **Axum**.

Para ORM y migraciones usar **SQLx + sqlx migrate**

Usar variables de entorno en .env

Endpoints requeridos

POST /api/v1/auth/register
 POST /api/v1/auth/login
 POST /api/v1/auth/refresh
 POST /api/v1/auth/logout
 GET /api/v1/health
 GET /api/v1/swagger-ui/

Principio	Cumplimiento
SRP	✓ Cada módulo hace una cosa
OCP	✓ Cambias infra sin tocar dominio
LSP	✓ Traits para repositorios
ISP	✓ Interfaces pequeñas
DIP	✓ Use cases dependen de traits

security_headers.rs y cors.rs

Excelente que no se mezclen en main.rs

Poblar la base de datos

Seed de datos

Crear usuarios de prueba

Usar el mismo repositorio real

Hashear passwords con Argon2

Respetar Clean Architecture

Enfoque elegido: Binario separado (src/bin/seed_dev.rs)

Esta es la mejor práctica en Rust:

Cargo soporta múltiples binarios

No mezclas seed con la API

Puedes ejecutar: cargo run --bin seed_dev

Estructura para datos seed

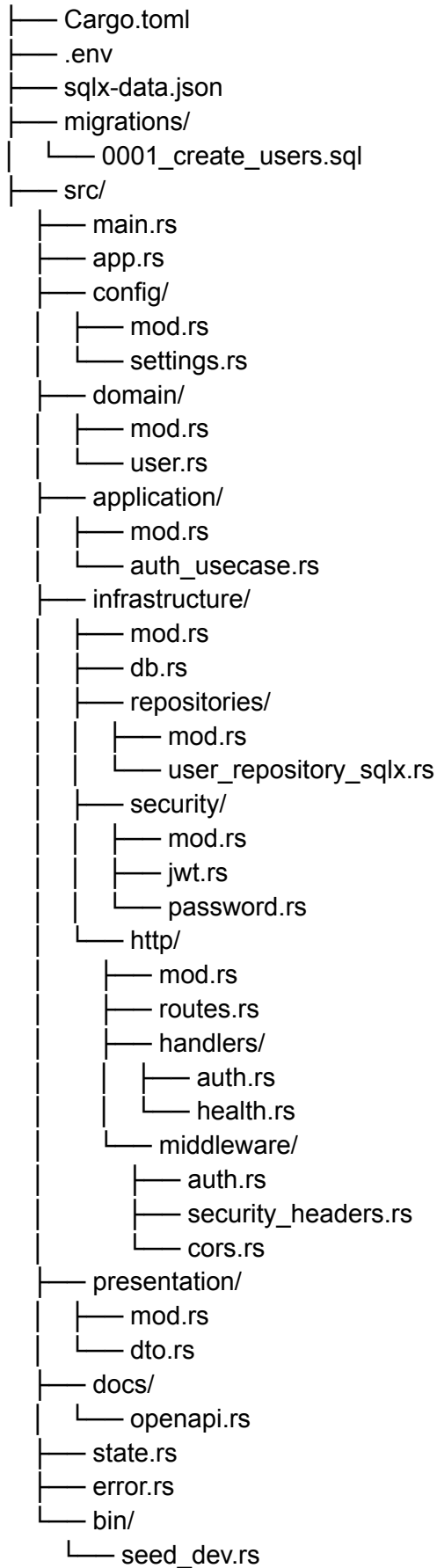
Ejecutarse solo en local

No contaminar migraciones ni la API

```
src/
├── bin/
│   └── seed_dev.rs
```

Estructura

auth-api-rust/



domain/

- User (entidad)
- Reglas puras
- NO serde
- NO SQLx
- NO HTTP

application/ (Aquí vive la lógica real)

- Casos de uso:
 - register_user
 - login_user
 - refresh_token
- Orquestan:
 - validaciones
 - hashing
 - repositorios

presentation/

DTOs con: `#[derive(Deserialize, Serialize, Validate, ToSchema)]`

Validaciones explícitas:

longitud, formato, required, password strength

infrastructure/

- SQLx
- JWT
- Cookies
- Middleware
- HTTP (Axum)

Todo lo reemplazable.

Bootstrap de la app

src/

```
|— main.rs
|— app.rs
```

main.rs → solo bootstrap (runtime, logging, start server)

app.rs → construcción del router, middlewares, state

Esto evita que main.rs se vuelva un monstruo.

Configuración

config/

```
|— mod.rs
|— settings.rs
```

- ✓ Separación clara
- ✓ Ideal para dotenv + envy
- ✓ Fácil de testear

Infrastructure (muy bien organizada):

repositories/

└─ user_repository_sqlx.rs

- ✓ Permite luego en el futuro:
 - user_repository_mock.rs
 - user_repository_redis.rs

Dependency Inversion en acción.

HTTP

http/

└─ routes.rs

└─ handlers/

└─ middleware/

Esto es muy limpio:

- Handlers → solo HTTP
- Middleware → auth, headers, cors
- Routes → wiring

Presentation

presentation/

└─ dto.rs

- ✓ DTOs + serde
- ✓ validator
- ✓ Sin lógica de negocio

Blindaje contra inputs maliciosos.

App State & Errors

state.rs → DB pool, config, jwt secrets

error.rs → errores de dominio + HTTP mapping

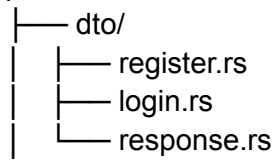
Resultado final

- API segura
- Arquitectura limpia
- Rendimiento nativo
- Seguridad enterprise
- Swagger integrado
- Escalable a microservicios

Sugerencia:

DTOs separados por intención (si crece en el futuro)

presentation/



Solo cuando el proyecto crezca.